

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	V Yogeswar	Reg. No.:	192425175
Section / Batch:	AI & ML / 2024	Date:	

Code of Conduct

I V Yogeswar / 192425175 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: V Yogeswar

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

Algorithm / Pseudocode

ALGORITHM ReferenceResolution(discourse)

1. Identify all named entities and noun phrases.
2. Identify all referring expressions such as pronouns.
3. For each pronoun:
 - a. Generate possible antecedents from previous discourse.
 - b. Check gender agreement.
 - c. Check number agreement.
 - d. Check recency.
 - e. Check semantic compatibility.
 - f. Check grammatical role and discourse coherence.
4. Assign a score to each valid antecedent.
5. Select the antecedent with the highest score.
6. Replace each pronoun with its selected antecedent.
7. Return the resolved discourse.

END ALGORITHM

Application to the Given Discourse

Discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Step 1: Entities and Referring Expressions

Entities:

- Meena
- Kavya
- laptop
- project

Referring expressions:

- she
- one
- her
- She
- her
- it

Step 2–4: Reference Resolution

Referring Expression	Possible Antecedents	Resolution Reason	
she	Meena, Kavya	Kavya	Both agree in gender and number, but context indicates Kavya needed the laptop.
one	laptop	laptop	one refers to the previously mentioned laptop.
her	Kavya	Kavya	Refers to Kavya's project.
She	Kavya	Kavya	Kavya is the recent discourse focus.
her	Meena, Kavya	Meena	Kavya thanked Meena, making Meena the recipient of the action.
it	laptop	laptop	it refers to the laptop being used.

Final Resolved Discourse

"Meena gave Kavya a laptop because Kavya needed a laptop for Kavya's project. Kavya thanked Meena and started using the laptop."

Handling Similar Antecedents

When multiple antecedents have similar gender and number properties, the algorithm does not rely only on agreement. It also considers **recency, grammatical role, semantic compatibility, and discourse coherence**. The antecedent with the highest combined score is selected, reducing incorrect resolution.

2. Algorithm for Entity Chains and Discourse Coherence

Algorithm / Pseudocode

ALGORITHM EntityChainAnalysis(discourse)

1. Divide the discourse into sentences.
2. Extract noun phrases and referring expressions.
3. Assign an entity ID to each new entity.
4. Compare each expression with previously identified entities.
5. Link expressions referring to the same entity.
6. Construct entity chains across sentences.
7. Count the occurrences of each entity in each sentence.
8. Measure entity continuity across consecutive sentences.
9. If important entities remain connected across sentences, classify the discourse as coherent.
10. Otherwise, identify breaks in entity continuity.

END ALGORITHM

Application

Discourse:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Step 1: Entities and Referring Expressions

- Anjali
- laptop
- processor
- Python
- computer
- machine learning application

Step 2–3: Entity Chains

Entity Chain 1: Anjali

Anjali → She → Anjali

Entity Chain 2: Laptop

laptop → The laptop → it → the computer

Entity Chain 3: Processor

processor

Entity Chain 4: Python

Python

Entity Chain 5: Application

machine learning application

Entity Continuity

Sentence Anjali Laptop

S1	✓	✓
S2	–	✓
S3	✓	✓
S4	✓	✓

The main entities **Anjali** and **laptop** remain connected throughout the discourse.

Conclusion

The discourse is **coherent** because references such as **She, it,** and **the computer** maintain links to previously introduced entities.

Breaking an entity chain can reduce coherence because the reader may no longer know which entity a pronoun or noun phrase refers to. This makes the discourse difficult to understand.

3. Algorithm for Identifying Discourse Relations

Algorithm / Pseudocode

ALGORITHM DiscourseRelationIdentification(discourse)

1. Divide the discourse into individual sentences.
2. Identify discourse markers in each sentence.
3. For each pair of consecutive discourse units:
 - a. Check for explicit markers.
 - b. Analyze the meaning of both units.
 - c. Determine the logical relationship.
4. Assign a discourse relation:
 - Cause-Effect
 - Sequence
 - Problem-Solution
 - Elaboration
5. Construct the discourse structure.
6. Return the identified relations.

END ALGORITHM

Application

Discourse:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Step 1: Discourse Units

DU1: The server crashed unexpectedly.

DU2: Users could not access the application.

DU3: The technical team restarted the server.

DU4: The system became available again.

Step 2–4: Relations

Discourse Units	Marker	Relation	Explanation
DU1 → DU2	As a result	Cause–Effect	Server crash caused users to lose access.
DU2 → DU3	None	Problem–Solution	Lack of access is the problem; restarting the server is the solution.
DU3 → DU4	After that	Sequence	System availability followed the server restart.

Discourse Structure

DU1: Server crashed



Cause–Effect



DU2: Users could not access application



Problem–Solution



DU3: Technical team restarted server

↓
Sequence

↓
DU4: System became available

Conclusion

The relations create a logical progression:

Problem → **Effect** → **Solution** → **Result**

Therefore, the discourse is logically coherent and easy to understand.

4. Algorithm for Dialogue Act Classification

Algorithm / Pseudocode

ALGORITHM DialogueActClassification(conversation)

1. Preprocess each utterance.
2. Tokenize the utterance.
3. Identify keywords, sentence patterns, and question forms.
4. Determine the communicative intention.
5. Assign one dialogue act:
 - Greeting
 - Request
 - Question
 - Inform
 - Offer
 - Confirmation
6. Generate the dialogue-act sequence.
7. Return the classified conversation.

END ALGORITHM

Application

User: "Hello, I need help with my assignment."

Dialogue Act: Greeting + Request

Reason: Hello indicates a greeting and I need help indicates a request.

Agent: "Sure, what problem are you facing?"

Dialogue Act: Question

Reason: The utterance asks the user for information about the problem.

User: "I do not understand machine learning."

Dialogue Act: Inform

Reason: The user provides information about their difficulty.

Agent: "I can explain the basic concepts to you."

Dialogue Act: Offer

Reason: The agent offers to provide an explanation.

Dialogue-Act Sequence

User → Greeting + Request

Agent → Question

User → Inform

Agent → Offer

Importance

Dialogue-act recognition helps a conversational agent understand the user's communicative intention and select an appropriate next response. For example, after an **Inform** act, the agent can provide an explanation, while after a **Question**, the agent can provide the requested information.

5. Algorithm for Dialogue State Tracking

Algorithm / Pseudocode

ALGORITHM DialogueStateTracking

1. Initialize all dialogue slots as UNKNOWN.
2. Read each user utterance.
3. Extract entities and values from the utterance.
4. Identify the corresponding slot.
5. Update the dialogue state.
6. Check which required slots are still UNKNOWN.
7. Select the next missing slot.
8. Generate an appropriate question.
9. Continue until all required information is collected.

END ALGORITHM

Step 1: Initial Dialogue State

Departure City = UNKNOWN

Destination City = UNKNOWN

Travel Date = UNKNOWN

Number of Passengers = UNKNOWN

Step 2: User

"I want to book a flight."

No slot value is provided.

Departure City = UNKNOWN

Destination City = UNKNOWN

Travel Date = UNKNOWN

Number of Passengers = UNKNOWN

Next question:

"Where would you like to travel?"

Step 3: User

"I want to go to Delhi."

Update:

Destination City = Delhi

Current state:

Departure City = UNKNOWN

Destination City = Delhi

Travel Date = UNKNOWN

Number of Passengers = UNKNOWN

Next question:

"From which city?"

Step 4: User

"From Chennai."

Update:

Departure City = Chennai

Current state:

Departure City = Chennai

Destination City = Delhi

Travel Date = UNKNOWN

Number of Passengers = UNKNOWN

Final Dialogue State

Slot	Value
Departure City	Chennai
Destination City	Delhi
Travel Date	UNKNOWN
Number of Passengers	UNKNOWN

Next Appropriate Question

"What is your travel date?"

Conclusion

Dialogue state tracking maintains information across multiple turns. It prevents the conversational agent from forgetting previously provided information and allows it to ask only for the missing details, resulting in a coherent and efficient multi-turn conversation.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1	3	3	3	3	3	15	75
2	3	3	3	3	3	15	
3	3	3	3	3	3	15	
4	3	3	3	3	3	15	
5	3	3	3	3	3	15	

Faculty In-charge	Course Coordinator	Program Director
KUMARAGURUBARAN. T		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____